



Constraint-Aware Pipeline for Generating High-Quality Utterances from OpenAPI Specifications Using LLMs

Vitor Gaboardi dos Santos¹, Boualem Benatallah²,
and Silvana Togneri MacMahon¹

¹ Dublin City University, Dublin, Ireland

{vitorgaboardidos.santos,silvana.macmahon}@dcu.ie

² Insight Research Ireland Research Centre on Data Analytics, School of Computing,
Dublin City University, Dublin, Ireland
boualem.benatallah@dcu.ie

Abstract. The integration of Large Language Models (LLMs) with APIs requires high-quality datasets that link user utterances to valid API calls. However, existing approaches for generating such datasets often overlook important quality attributes, leading to unnatural utterances, limited parameter coverage, and invalid API calls that violate documentation constraints. In this paper, we present a constraint-aware pipeline (CAP) for generating high-quality utterance-API call pairs from OpenAPI Specifications (OAS). CAP explicitly models and incorporates API constraints during generation to ensure constraint adherence, naturalness, and parameter diversity. We generate synthetic datasets using different LLMs and compare CAP against an existing prompting-based pipeline. Experimental results show that CAP improves constraint adherence, parameter coverage and naturalness.

Keywords: LLM · API · Utterances · Chatbots · Constraints

1 Introduction

Large Language Models (LLMs) have shown strong performance in conversational AI, serving as natural language interfaces that fulfil user utterances and enable interaction with services through the integration with tools and APIs [3]. Nevertheless, training or fine-tuning robust models to perform API selection or API calling requires high-quality datasets linking utterances to APIs [4, 8].

Recent work has focused on generating utterance-API call pairs using OpenAPI specification (OAS). ToolBench [9] generated utterances with different levels of complexity. Sheng et al. [12] generated datasets to evaluate an LLM's proficiency in using APIs by designing a prompt with guidelines encouraging parameter diversity and natural values for parameters. However, existing work often overlooks different quality attributes of datasets simultaneously, which have shown to impact the performance of tool-using LLMs trained on such data [4].

In this paper, we propose a pipeline for generating high-quality datasets of utterance-API call pairs, focusing on three quality attributes: *naturalness*, *parameter diversity*, and *constraint adherence*. *Naturalness* ensures that utterances sound like authentic human requests when talking to chatbots [13]. *Parameter diversity* ensures the generation of utterances with different parameters and their combinations, reflecting distinct user intents [10]. *Constraint adherence* ensures that API calls include parameters that respect constraints imposed by OAS, such as valid values, formats, and dependencies between parameters, making utterances closely aligned with the API method [2, 12]. For instance, a “*departureDate*” parameter may require a YYYY-MM-DD format and dates only after 2022-01-01. Failing to address these attributes may result in utterances and API calls that do not generalize to real-world conversations, fail to generalize to parameter combinations, and produce utterances with infeasible API calls [4, 12].

In summary, we make the following contributions: (1) we propose a constraint-aware pipeline (CAP) to generate natural and parameter-diverse utterances that adhere to OAS constraints; and (2) we generate datasets using GPT-4o and DeepSeek-V3 with CAP and compare their quality against a pipeline proposed by Sheng et al. [10]. The code, dataset, and results are available on GitHub¹.

2 Constraint-Aware Pipeline for Utterance-API Call Pairs Generation

As summarised in Fig. 1, our pipeline consists of three steps: (1) API parameter constraint extraction; (2) OAS enrichment; and (3) utterance-API call pairs generation.

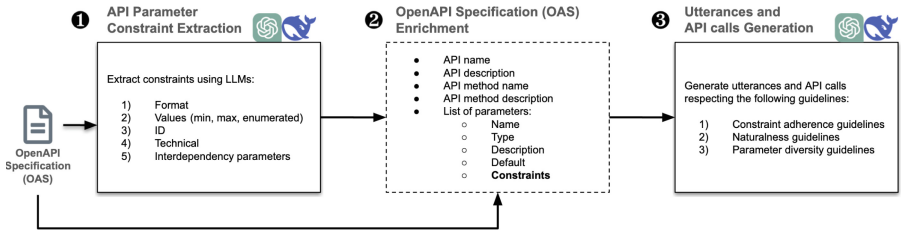


Fig. 1. Pipeline overview for generating constraint adherence, natural, and parameter diverse utterance-API call pairs using LLMs from OpenAPI specifications.

¹ <https://github.com/vitorgaboardi/ConstraintAPIBench>.

2.1 API Parameter Constraint Extraction

CAP extracts five types of parameter constraints from OAS. We build upon standard constraint definitions from API testing literature [1, 5, 6], while introducing a Technical constraint to filter non-conversational parameters and enhance the generation of natural utterances. The five constraint types are defined as follows: (1) **Values constraint (VAL)**: Possible values that a parameter can assume. This includes: Maximum (maximum value); minimum (minimum value); and enumerated (a closed set of acceptable values); (2) **Format constraint (FOR)**: Any required input format a parameter must adhere to in an API call, such as IATA airport code (e.g., MAD for Madrid) or ISO currency code (e.g., USD for US dollar); (3) **ID constraint**: Boolean value that is True if the parameter represents an identifier (ID) (e.g., message_id, gameId, hotelIds). (4) **Technical constraint (TEC)**: Boolean value that is True if the parameter is primarily used by developers rather than end users for creating API calls or formatting pages (e.g., parameters such as page number, page size, offset, access token, API key). Parameters used by users that affects sorting or limits the output must be False (e.g., sort, limit, orderBy, sortBy). (5) **Interdependency constraints (IDP)**: Describes dependencies between parameters. We use six constraints presented in Martin et al. [5]. They refer to a set of parameters and include constraints where: (5.1) the presence of one parameter requires the presence of another parameter (e.g., if startDate is provided, endDate must be included); (5.2) one or more parameters must be included in the API call (e.g., at least one contact method, such as email or phone, must be provided); (5.3) one and only one parameter must be included in the API call (e.g., either username or email must be provided in an authentication API); (5.4) either all or none parameters must be included in the API call (e.g., latitude and longitude must be included only together); (5.5) zero or one parameter can be present in the API call (e.g., at most one discount method, such as promoCode or discountId, can be provided); (5.6) parameters are related by arithmetic constraints (e.g., minPrice must be less than maxPrice).

To ensure that the LLM can identify constraints even when they are not mentioned in parameter descriptions, we provide to the LLM the API name, API description, API method name, API method description, and a list of all parameters with their name, type, description, default value, and whether it is required or not. Finally, we include one example of a flight search API method with ten parameters that illustrate at least one instance of each constraint type.

2.2 OpenAPI Specification (OAS) Enrichment

CAP enriches an original OAS in two ways to provide a more complete representation of API parameter constraints. First, a “constraint” field is added to each API parameter description in the enriched OAS, listing all constraints extracted in the previous step. Making constraints explicit in the OAS offers a well-structured reference that supports the generation of utterance-API call pairs

with better constraint adherence. Second, parameters where the “technical” constraint was True are removed in the enriched OAS. These parameters typically represent control configurations intended for developers, making it unnatural to include them in utterances. For instance, a user would not normally say mention “api key” information.

2.3 Utterance-API Call Pairs Generation

CAP generates utterance-API call pairs by prompting LLMs with the enriched OAS, which explicitly details parameter constraints. The pipeline guides the generation process using structured rules designed to ensure constraint adherence, naturalness, and parameter diversity.

Constraint Adherence. We proposed four rules to ensure constraint adherence: (1) required parameters must be included in all generated utterance-API call pairs; (2) values must conform to range limits and respect the enumerated set of values; (3) the parameter values in the API call must conform to the format constraint; and (4) the combination of parameters must respect any interdependency constraints. The generated utterance-API call pairs will closely align with the corresponding API method if the LLM adheres to these rules, which likely reduces API calling errors.

Naturalness. We proposed five rules to ensure naturalness: (1) omit the API name from utterances, unless it corresponds to a well known consumer brand (e.g., Spotify, YouTube, Booking.com), rather than a generic API (e.g., Movies API, Converter API). (2) do not use general placeholders such as “this URL” or “this name”; (3) every parameter value in the API call must be inferable from the utterance; (4) do not include parameter values in the utterance with a format the user would not normally say. For instance, users say city names instead of latitude and longitude coordinates; (5) if the “ID” constraint is True, do not generate artificial IDs, but generate values representing what the ID refers to. IDs are commonly used to reference resources and are not known by users.

Parameter Diversity. We proposed five rules to promote parameter diversity: (1) include each optional parameter at least once across generated utterances, unless the number of optional parameters is too large for the number of utterances requested. This promotes parameter coverage while avoiding overloaded requests; (2) use different combinations of optional parameters across utterances. This reflects the many intents users might express a request. For instance, a movie search API might be called with parameters such (title, year) or (title, genre); (3) cycle through allowed values for any parameter that includes the “enumerated” constraint; and (4) avoid repeating parameter values across utterances. Rules (3) and (4) aim to ensure a rich set of parameter values.

3 Experiments

We designed experiments that assess the quality of generated utterances.

3.1 Dataset Generation

OAS Dataset. We selected a subset of 237 APIs (1,044 methods) from Tool-Bench [9] to generate utterance-API call pairs. From each of the 49 categories, we selected the 5 most popular APIs based on the *popularity score* provided by RapidAPI. We applied three filtering criteria to select APIs: (1) each method must include at least one parameter; (2) API descriptions must contain at least five words; and (3) the number of methods must not exceed 30. These criteria were designed to ensure that parameter constraints can be extracted, to keep the dataset size manageable and to maintain a balanced distribution of APIs. Nevertheless, this design choice affects only evaluation, not the applicability of CAP, which can be applied to large industrial APIs with hundreds of operations.

Data Generation Pipelines. We generated 10 utterance-API call pairs per API method using two pipelines: (1) our proposed **constraint-aware pipeline (CAP)**, which implements the design principles described in Sect. 2; and (2) the **description-based pipeline (DBP)**, based on Sheng et al. [10]. We selected DBP because it has instructions that align with our goals: promoting naturalness, covering all parameters, and requiring parameter inclusion.

Models. We used GPT-4o and DeepSeek-V3 models to generate utterance-API call pairs. We set the temperature to 0.0 for reproducibility.

3.2 Evaluation Metrics

We evaluate the quality of generated utterances across three dimensions: naturalness, parameter diversity, and constraint adherence. We report metrics on a sample of 50 randomly selected APIs, which led to 237 API methods.

Naturalness. We evaluate naturalness using two LLMs as judges: Llama-3.1-70B and GPT-4.1. Recent studies have shown that LLM-based evaluations have high correlation with human judgments when evaluating text quality attributes [7]. We provide LLMs with a definition of naturalness, examples of utterances, and ask the LLM if each utterance is ‘natural’ or ‘unnatural’, reporting the number of utterances labelled as ‘natural’.

Parameter Diversity. We measure parameter diversity using two metrics: **Parameter Coverage (PC)**, which represents the proportion of an API method’s parameters used across generated utterances, and **Parameter Combination Coverage (PCC)**, which represents the diversity of parameter combinations in generated API calls by counting unique parameter combinations per method. Higher PCC values indicate broad intent coverage.

Constraint Adherence. We measure constraint adherence by counting violations using an automated checker. It validates generated parameter values against ground truth constraints (manually annotated from the OAS) for three types: (1) Values (VAL): Checks if values fall within numeric ranges or belong to enumerated sets. (2) Format (FOR): Uses regular expressions to verify string patterns. (3) Interdependency (IDP): Evaluates logical conditions between parameters. A violation occurs when any check fails. We focus on these types because they evaluate whether values respect API requirements, while ID and technical constraints guide parameter inclusion during generation.

3.3 Results

We compare quality performance in terms of naturalness, parameter diversity, and constraint adherence between generation pipelines and LLMs.

Naturalness. We found that **CAP generated more natural utterances than DBP for both LLMs, and DeepSeek-V3 created more natural utterances than GPT-4o.** Table 1 shows the number of utterances judged as natural from Llama-3.1-70B (NAT-LL3.1) and GPT-4.1 (NAT-GPT4.1) as well as the Cohen’s κ inter-rater agreement coefficient [11] between these two models.

Table 1. Naturalness comparison between DBP and CAP across different LLMs. Values show the percentage of utterances labeled as natural.

Model	Prompt	NAT-LLA3.1	NAT-GPT4.1	Cohen’s κ
GPT-4o	DBP	1,080 (50.7%)	991 (46.5%)	80.2%
	CAP	1,384 (67.1%)	1,293 (62.7%)	75.4%
DeepSeek-V3	DBP	967 (44.3%)	930 (42.6%)	80.5%
	CAP	1,479 (67.5%)	1,526 (69.6%)	75.1%

CAP improved naturalness by 16.4% for GPT-4o and by 23.1% for DeepSeek-V3 in the NAT-LLA3.1 evaluation, demonstrating that CAP consistently guides LLMs towards more natural generation. Through CAP, DeepSeek-V3 achieved higher naturalness than GPT-4o (69.7% vs. 62.8%) according to NAT-GPT4.1, indicating that CAP has a stronger positive impact on naturalness for DeepSeek-V3. While NAT-GPT4.1 showed lower scores overall (suggesting stricter criteria), the relative ranking remained consistent. Furthermore, GPT-4o with CAP generated the natural utterance *Can you suggest some songs similar to ‘Shape of You’ by Ed Sheeran?* while DBP generated the unnatural *Find similar songs to the one identified by key 987654321*, referencing the song by ID.

To validate the LLM-as-a-judge evaluation, we randomly sampled 100 utterances per configuration (400 total) and had one author manually label each as “natural” or “unnatural.” We computed Cohen’s κ for each configuration, achieving substantial inter-rater agreement: $\kappa = 0.70 \pm 0.07$ (vs. GPT-4.1) and $\kappa = 0.66 \pm 0.06$ (vs. Llama-3.1-70B). These results support the use of LLM judges for naturalness evaluation, despite some misclassifications.

Parameter Diversity. We found that **CAP achieved higher parameter coverage than DBP for both models. CAP also produced more unique parameter combinations with GPT-4o, while DeepSeek-V3 produced fewer parameter combinations.** Table 2 shows the parameter coverage (PC), parameter combination coverage (PCC) and total parameters (TP).

Table 2. Parameter diversity results using DBP and CAP across different LLMs.

Model	Prompt	PC	PCC	TP
GPT-4o	DBP	0.9092	373	592
	CAP	0.9823	416	545
DeepSeek-V3	DBP	0.9140	416	592
	CAP	0.9733	376	499

CAP improved parameter coverage for GPT-4o from 90.92% to 98.23%. CAP also increased unique parameter combinations from 373 to 416 despite reducing the total parameter pool from 592 to 545 (47 fewer technical parameters). For DeepSeek-V3, CAP achieved higher parameter coverage than DBP (97.33% vs. 91.40%), but decreased parameter combination coverage from 416 to 376. This reduction is mainly due to the decrease in the total parameter pool from 592 to 499 (93 fewer technical parameters were filtered by CAP), limiting the number of possible parameter combinations.

Constraint Adherence. We found that **CAP reduced constraint violations for both models compared to DBP, and GPT-4o produced fewer violations than DeepSeek-V3**. Table 3 reports constraint violations in terms of Values (VAL), Format (FOR), and Interdependency (IDP) constraints. Results are reported across the same set of 50 evaluation APIs.

Table 3. Constraint violations count using DBP and CAP across different LLMs. Lower values indicate better constraint adherence.

Model	Prompt	VAL	FOR	IDP	Total
GPT-4o	DBP	10	49	7	66
	CAP	2	21	4	27
DeepSeek-V3	DBP	26	59	6	91
	CAP	19	34	0	53

CAP reduced total violations by 59% for GPT-4o (66 to 27) and by 42% for DeepSeek-V3 (91 to 53), demonstrating that constraint-aware generation improves API compliance. The largest improvements came from FOR violations, which decreased from 49 to 21 for GPT-4o and from 59 to 34 for DeepSeek-V3. CAP also eliminated all IDP violations for DeepSeek-V3 (0 vs. 6 with DBP), although GPT-4o still had minimal violations under both conditions.

This paper presented a constraint-aware pipeline for generating high-quality utterances from OAS using LLMs. Our approach models different constraint types and demonstrated improved constraint adherence, naturalness, and parameter diversity, indicating the potential of constraint-aware generation for building

reliable API-grounded language interfaces. Future work will focus on extending the framework to additional constraint categories, evaluating more LLMs and pipelines, and fine-tune LLMs using the generated data.

Acknowledgments. This publication has emanated from research conducted with the financial support of Taighde Éireann-Research Ireland under Grants No. 18/CRT/6223 and SFI/12/RC/2289_P2.

References

1. Alonso, J.C., Martin-Lopez, A., Segura, S., Bavota, G., Ruiz-Cortés, A.: Satori: Static test oracle generation for rest apis (2025). arXiv preprint
2. Alonso, J.C., Martin-Lopez, A., Segura, S., Garcia, J.M., Ruiz-Cortes, A.: Arte: automated generation of realistic test inputs for web apis. *IEEE Trans. Software Eng.* **49**(1) (2022)
3. Dam, S.K., Hong, C.S., Qiao, Y., Zhang, C.: A complete survey on llm-based ai chatbots (2024). arXiv preprint
4. Iskander, S., Tolmach, S., Shapira, O., Cohen, N., Karnin, Z.: Quality matters: Evaluating synthetic data for tool-using LLMs. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pp. 4958–4976, Miami, Florida, USA (2024)
5. Martin-Lopez, A., Segura, S., Ruiz-Cortés, A.: A catalogue of inter-parameter dependencies in restful web apis. In: *Service-Oriented Computing: 17th International Conference, ICSOC 2019, Springer, Toulouse, France* (2019)
6. Martin-Lopez, A., Segura, S., Ruiz-Cortés, A.: Restest: Black-box constraint-based testing of restful web apis. In: *Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates. Springer* (2020)
7. Murugadoss, B., Poelitz, C., Drosos, I., Le, V., McKenna, N., Negreanu, C.S., Parnin, C., Sarkar, A.: Evaluating the evaluator: Measuring llms’ adherence to task evaluation instructions. *Proceedings of the AAAI Conference on Artificial Intelligence* **39**(18), pp. 19589–19597 (2025)
8. Patil, S.G., Zhang, T., Wang, X., Gonzalez, J.E.: Gorilla: large language model connected with massive apis. In: *Advances in Neural Information Processing Systems*, vol. 37, pp. 126544–126565 (2024)
9. Qin, Y., et al.: Toolllm: facilitating large language models to master 16000+ real-world apis. arXiv preprint [arXiv:2307.16789](https://arxiv.org/abs/2307.16789) (2023)
10. Sheng, Y., Gandhe, S., Kanagal, B., Edmonds, N., Fisher, Z., Tata, S., Selvan, A.: Measuring an llm’s proficiency at using apis: a query generation strategy. In: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5680–5689. KDD ’24, New York, NY, USA (2024)
11. Sun, S.: Meta-analysis of cohen’s kappa. *Health Serv. Outcomes Res. Method.* **11**(3), 145–163 (2011)
12. Wang, S., Jean, S., Sengupta, S., Gung, J., Pappas, N., Zhang, Y.: Measuring and mitigating constraint violations of in-context learning for utterance-to-API semantic parsing. In: *Findings of the Association for Computational Linguistics*, pp. 7196–7207, Singapore (2023)
13. Yaghoub-Zadeh-Fard, M.A., Benatallah, B., Casati, F., Barukh, M.C., Zamanirad, S.: User utterance acquisition for training task-oriented bots: a review of challenges, techniques and opportunities. *IEEE Internet Comput.* **24**(3) (2020)